

Rename network interfaces based by MAC address using .link file

CUSTOM SCHEMES USING .LINK FILES

there's a canonical way of overriding the default name of network interfaces: you can use `.link` files to set up naming policies to suit your needs. Thus for instance if you have two PCs each of which has only a single wireless card, but one calls it `wlp0s1` and the other `wlp1s0`, you can arrange for them both to use the name `wifi0` to simplify sharing firewall configurations. For details see [systemd.link\(5\)](#).

Here's a relatively futureproof "manual" version:

- STEP 1: create a `.link` file..... `10-filename.link` in `/etc/systemd/network`
- STEP 2: if necessary (most likely yes), edit `etc/NetworkManager/NetworkManager.conf` to disable mac address randomization

```
#/etc/systemd/network/10-persistent-net.link

[Match]

MACAddress=01:23:45:67:89:ab


[Link]

Name=wlan0
```

Note: per [systemd.link\(5\)](#), you shouldn't use a name that the kernel might use for another interface (for example "eth0").

It is also possible to reorganize the naming policy by overriding `/lib/systemd/network/99-default.link`, for instance to insist that all network interfaces are named purely by MAC address:

```
#/etc/systemd/network/99-default.link

[Match]

OriginalName=*


[Link]

NamePolicy=mac

MACAddressPolicy=persistent
```

NOTE: IMPORTANT ! :: NetworkManager changes/randomizes certain network interfaces mac address automatically/ by default

For privacy reasons, the default settings for network manager cause the MAC address of the Wifi device to change constantly. To fix this, you just have to create a new config file (or edit the existing config file) as outlined below and then restart the NetworkManager service.

Randomization during Wi-Fi scanning NetworkManager by default

During Wi-Fi scanning, NetworkManager resets the MAC address frequently to a randomly generated address or/and after reboots:

To disable this behavior Edit your `/etc/NetworkManager/NetworkManager.conf` as follows:

```
[device]
wifi.scan-rand-mac-address=no
```

Randomization during Wi-Fi scanning is enabled by default, but it may be disabled by adding the following lines to `/etc/NetworkManager/NetworkManager.conf` or a dedicated configuration file under `/etc/NetworkManager/conf.d`

```
[device]
wifi.scan-rand-mac-address=no
```

Setting it to `yes` results in a randomly generated MAC address being used when probing for wireless networks.

NetworkInterfaceNames

See below for a ["Predictable Names" Migration HOWTO](#).



Anything that changes the names of your network interfaces may result in the machine suddenly not being reachable over SSH, so if you're editing settings on a remote server, plan your changes carefully and doublecheck your safety nets.

This page deals with the various schemes by which wired and wireless network interfaces are assigned names - that is, the underlying system labels like `eth0` or `wlx800e1319c734`. It has nothing to do with the "connection profile" names used by apps such as [NetworkManager](#), like "Wired connection 1".

Contents

1. [THE ORIGINAL SIMPLE SCHEME](#)
 1. [Why it was abandoned](#)
 2. [How to get it back](#)
2. [THE "PERSISTENT NAMES" SCHEME](#)
 1. [Why this one was abandoned](#)
 2. [How to cling to it for now](#)
 3. [How to let go and move on](#)
3. [MISCELLANEOUS OLD SCHEMES](#)
4. [THE "PREDICTABLE NAMES" SCHEME](#)
 1. [How to cope with it on fresh installs](#)
 2. [How to migrate to this scheme on upgraded systems](#)
 3. [Complications and corner cases](#)
 4. [Safety nets](#)
5. [CUSTOM SCHEMES USING .LINK FILES](#)
6. [EXTERNAL REFERENCES](#)

THE ORIGINAL SIMPLE SCHEME

Back in the nineties, `eth0`, `eth1`, etc were simply assigned by the kernel.

Why it was abandoned

At least in theory, if module probes completed in a different order, `eth0` and `eth1` might switch places on successive boots. As boot processes became less linear and interfaces became more hotpluggable this became more of a concern.

How to get it back

If you wipe out all other name-assignment mechanisms then you'll be left with this one.

The simple way of disabling the whole [current interface naming scheme](#) (which you might want to try for one-off testing) is just to boot with the kernel parameter `net.ifnames=0`, which can be set in an interactive `grub` session at boot or made persistent by editing `/etc/default/grub` and running `update-grub`.

Alternatively, you can override `/lib/systemd/network/99-default.link`, with a custom version in `/etc/systemd/network/`, or similarly override `/lib/udev/rules.d/80-net-setup-link.rules`, or mask the latter by using a `/dev/null` symlink instead of a custom version, or... there seem to be lots of ways of doing this, so make sure you haven't done it in more than one way or it'll trip you up in a couple of years when you try to undo it. See the [external links](#) below on standard methods for overriding `systemd` configuration. Oh, and beware of [initrd skew](#).

(If you've still got a working [legacy](#) `70-persistent-net.rules` file, the `net.ifnames=0` flag doesn't deactivate that, so you'd need to see [the instructions below](#) for temporarily disabling that file. It's probably also possible to do this by masking enough of `systemd`.)

THE "PERSISTENT NAMES" SCHEME

This scheme, introduced somewhere around Debian 5 "lenny", used `udev` to identify interfaces by MAC address and assign a fixed interface number to any interface it recognized (writing the rules to `/etc/udev/rules.d/70-persistent-net.rules`). This could have annoying side-effects (e.g. if you were replacing a machine's sole NIC, you'd also have to take special care to ensure it took over as network interface number 0) but these were minor and predictable.

A sample `.rules` entry:

```
#/etc/udev/rules.d/70-persistent-net.rules
# PCI device 0x8086:0x100e (e1000e)
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}=="01:23:45:67:89:ab", ATTR{type}=="1", KERNEL=="eth*", NAME="eth0"
```

Since Debian 9 "stretch", newly installed machines no longer start with an `/etc/udev/rules.d/70-persistent-net.rules` file, though such files are maintained if they still exist (with new lines added for newly installed network hardware). Debian 10 "buster" additionally lacks the `/lib/udev/rules.d/75-persistent-net-generator.rules` file that appends to it, though `legacy` `70-persistent-net.rules` files are still honored. This support is expected to be removed from Debian 11 "bullseye" at some time before it is released.

Why this one was abandoned

This still had subtle race conditions, required `/etc` to be on a writable file system, and had problems with virtualization, so it's no longer supported upstream; the plan (still taken for granted in most of the documentation) was for it not to be supported in Debian 10 "buster", but in the end it has scraped through into another release cycle.

How to cling to it for now

If you've got a working "legacy" `/etc/udev/rules.d/70-persistent-net.rules` file and want to stick with it, you can safely upgrade through Debian 9 "stretch" and Debian 10 "buster"; `udev` on these releases still respects that file if present (and will accept a freshly created one). However, bear in mind that on buster you'll need to maintain its contents yourself, and eventually even this legacy support will end, so you should be ready to switch to a different scheme before that happens.

How to let go and move on

If you're currently running something [newer than Debian 8 "jessie"](#) with a legacy `70-persistent-net.rules` file but have decided to switch to the [new regime](#), you can do that just by disabling the `.rules` file (then [updating the initrd](#) before you reboot); see the `udev` README.Debian.gz and [the more detailed guide below](#).

MISCELLANEOUS OLD SCHEMES

(Contributions welcome)

Several workarounds for renaming interfaces grew up in the early days of hotpluggable wireless interfaces, but if they still work it'll be because like  [ifrename](#) they now use `udev` rules under the hood. It's not clear what remaining advantage this has over [the canonical link approach](#) - is it perhaps useful for non-systemd machines?

Old releases of RedHat (among others) used a "biosdevname" system, but that's never been supported under Debian. If you need to know about it there's bound to be documentation somewhere.

THE "PREDICTABLE NAMES" SCHEME

The new scheme uses names usually derived from the location of the interface in terms of hardware buses etc: `enol, wlp1s3`. The idea was that this provides "Predictable Names", though as it turns out the main thing that's predictable about it is that calling it this will cause furious users to pop up disputing the appropriateness of that name. (Can we just skip all that here, please?)

How to cope with it on fresh installs

This should be easy enough; before you start configuring firewalls etc., just look at (e.g.) the output of `ip a` and note the names of the interfaces. Unlike the old days, when the only way to guess which cable was plugged into `eth0` and which was `eth1` was to keep track of MAC addresses, this system provides extra clues in the interface names.

How to migrate to this scheme on upgraded systems

It's advisable to do this as a separate migration in its own right, not as part of a general distribution upgrade. However, if your PC only has one network interface and not much is at stake you can try:

Strategy A

- wait until `udev` breaks your networking, if it's going to, or [trigger](#) the change yourself.
- ask `ip` what new name it's using, and fix your configuration files.

You should probably at least check in advance to see what files hard-code interface names, by running something like

```
sudo rgrep wlan0 /etc
```

Obvious likely hits include `/etc/network/interfaces` and configuration files for firewalls, wifi, DHCP... but it's possible that (e.g.) a laptop with a single wifi interface managed by NetworkManager might need no fixing at all.

Oh, and hang on, aren't there apps that want you to put per-interface configuration into a file *named after the interface*, like `/etc/whatever/wlan0.conf`? You might need to check for those, too:

```
sudo find /etc -name '*wlan0*'
```

Strategy B

This strategy, more or less compulsory for remote servers, runs along the lines of:

- devise a migration plan
- consider a dead-man's-handle `etckeeper` cronjob [or the like](#)
- consult the list of [corner cases](#) below and search the Internet for others
- once you're sure you're safe, implement your migration plan
- you've got backups, right?

To find out what names `udev` would be choosing between if you switched over to the new system, first get a list of the network devices the system knows about:

```
echo /sys/class/net/*
```

For each device path (other than `/sys/class/net/lo`), ask `udevadm` what `NET_IDs` it knows:

```
udevadm test-builtin net_id /sys/class/net/enp0s1 2>/dev/null
```

It's likely to tell you about things like `ID_OUI_FROM_DATABASE` and an `ID_NET_NAMING_SCHEME`, but the lines that matter are the ones (given in random order) starting with `ID_NET_NAME_`. One of these is the name that `udev` will give priority to - the list of candidates may be so short that all you need to know is that `..._PATH` beats `..._MAC`, but there are also some rarer possibilities, and in general if something unusual shows up then it will take priority.

From highest priority to lowest, the list is:

1. **ID_NET_NAME_FROM_DATABASE**= Very rare and not to be confused with `ID_OUI_FROM_DATABASE`; if present, it outranks any other `ID_NET_NAME`. The database is hardcoded into `udev` and has only one known entry, the spooky-sounding `idrac`.
2. **ID_NET_NAME_ONBOARD**= Appears for some but not all kinds of onboard network card - it's usually a nice simple name like `eno0` or `wlo0`.
3. **ID_NET_NAME_SLOT**= Appears for some PCI-hotplug cards. Usually looks like `ens0` or `wls0`. (Does this ever occur alongside `_ONBOARD`?)
4. **ID_NET_NAME_PATH**= Always present; usually something just complicated enough to be easy to forget, like `wlp3s5` or `enp1s3f0`. Note that all numbers are in hex.
5. **ID_NET_NAME_MAC**= Also always present, but with a low enough priority that by default it won't be used; e.g. `wlx800e1319c734`

One good reason for doing the migration separately from a dist-upgrade is that the answers you get from (e.g.) stretch's `udevadm` aren't absolutely guaranteed to be identical to the answers you get from buster's. This further implies that any time you update `systemd` and reboot there's a chance your server might fall off the net, which seems like a good argument for using a [customized scheme](#) at least for the interface you're SSHing in on.

Complications and corner cases

(Additions welcome, but please try to avoid ballooning this section with tales of "I don't know how this happened but it all went wrong for me"...)

* ANTIQUE SYSTEMS

on Debian 9 "stretch" or newer, merely booting without a `net.ifnames=0` override (and without a `70-persistent-net.rules` file) should be enough to let you run the new scheme, but on Debian 8 "jessie" you'll need to actively set it to `net.ifnames=1`.

* TRIGGERING THE SWITCH

the brave way of finding out what network interface names you'll get without a `/etc/udev/rules.d/70-persistent-net.rules` file is to delete it (and update your [initrd](#) before you reboot), but you don't need to go that far. Just renaming it (e.g. to `70-persistent-net.rules.old`) or commenting out particular lines should be enough. See the `udev` [README.Debian.gz](#) file. Note that it is possible to have a mixed system with (say) an `enp1s1` named from its hardware path alongside a `wlan0` still defined as a "persistent" name.

* THE ID_NET_NAME_ HIERARCHY

if you're ignoring `ID_NET_NAME_SOMETHING` on the assumption that anything you don't understand probably isn't important, you need to reread [the above](#) - the general rule is, if you don't recognise it, it'll mess things up.

* PREFIXES AND PATHS

wired devices get a prefixed `en-` for Ethernet, wireless ones get `wl-` (and there are also a few more obscure possibilities such as `ib-` for InfiniBand); then in principle it's possible to decipher all the following [sequences of code letters plus hex digits](#) that encode hardware topology. But there's not much point trying to learn all the details, since the only workable way of predicting what `ID_NET_NAMEs` an interface will get is to ask `udevadm`, which will tell you the full strings.

* MYSTERY PRIORITIES

if you look at `/lib/systemd/network/99-default.link` on buster, you'll see that the standard priority hierarchy goes "keep kernel database onboard slot path" (ordered from highest to lowest; `mac` only gets considered via a different mechanism). There's a distinct shortage of documentation for those first three name-types, but the best source for `keep` (post-stretch) is `/usr/share/doc/systemd/NEWS.gz` (n.b. **not** `NEWS.Debian.gz`), which explains that it was formerly treated as present by default, and now exists as an explicit rule that names assigned by [custom .link files](#) won't be overridden. The name-type `kernel` means something similar for interface names that have been "declared as persistent", but it's unclear what this is talking about.

* USB DEVICES

since they might get plugged into a different socket each time, these use `ID_NET_NAME_MAC` - automated via `/lib/udev/rules.d/73-usb-net-by-mac.rules`.

* VIRTUAL MACHINES

on virtual machines (according to the `udev` [README](#)) you will need to remove the files `/etc/systemd/network/99-default.link` and (if using virtio network devices) `/etc/systemd/network/50-virtio-kernel-names.link`, then [rebuild the initrd](#).

* INITRD SKEW

it's all very well having everything sorted out in `/etc`, but interface renaming has to happen very early during boot; to make sure your `initrd` doesn't contain out-of-date versions of important `systemd` files, regenerate it with `sudo update-initramfs -u`

* LEFTOVERS

the old persistent-names system first started being publicly deprecated in `NEWS` files back in mid-2015, so this upgrade has been hanging ominously over people's heads for a long time. Are you sure you didn't do something about it the last time the subject came up, like setting up a `net.ifnames=0` kernel parameter, and/or masking some `systemd` config file? If so, this may result in confusing symptoms when you try to go over to the new system. Check your administrative logbooks. What do you mean, you don't keep logs?

* BUGS

users of  [jwld](#) in bullseye currently need to be aware of bug  [#944097](#): wifi interfaces may be brought up before `udev` can rename them.

* TO BE DETERMINED

if you're aware of extra sources of complications not accounted for here involving (for instance) non-systemd init systems; minor ports; `systemd-networkd`; or something else that has turned up since this was first written, please add them here.

* UNPREDICTABILITY

it turns out even after all this there are still reported cases of interfaces changing their name on a reboot. All that needs to happen is that some buggy BIOS (or some new, less buggy version of a driver module, or systemd's naming policy) changes its mind about some detail like whether or not your hardware counts as the kind that should have an ONBOARD name. There are even  [reports](#) of devices changing their PCI-port numbering due to other hardware being installed.

Safety nets

* DEAD-MAN'S-HANDLE

if you're rebooting with `70-persistent-net.rules` [renamed as](#) `70-persistent-net.rules.old`, and there's a danger that you might find yourself locked out, you can set up a precautionary script (called by `/etc/rc.local`, `@reboot` cronjob, or systemd timer) that waits a few minutes, then copies the file back again, rebuilds the `initrd`, and reboots. Extra fancy checks and beeping noises and so on are possible, but increase the risk of failure.

* HOKEY-COKEY MIGRATION

if there are two interfaces, either of which could be used to give you remote access, both [currently named via](#) `70-persistent-net.rules`, you can comment one of them out of the file, [update the `initrd`](#), reboot to see what it's called now, then go back and reactivate it before commenting out the other. If everything's still going okay, you can finish it all off.

* CATCHALL INTERFACE NAMING

if your SSH interface is expected to come back as `enp0s1` after the reboot, and that's what you've got configured in `/etc/network/interfaces`, but instead it decides to call itself `eno0`, that's a problem - but one that a sufficiently cautious admin can guard against by having entire duplicate stanzas in the `interfaces` file to define the same IP address for every name it might plausibly come back with, including `eno0`, `ens0`, `eth0`, and so on. Mind you, this still won't help if it comes back as `enp7s1`.

* MANUAL OVERRIDE

one way of being sure is to avoid trusting `udev` to make its own mind up about what your crucial network interface should be called; switch it over to a name [defined](#) in a custom `.link` file.

* EXTRA SUGGESTIONS GO HERE

...

CUSTOM SCHEMES USING .LINK FILES

The scheme detailed above is the new standard default, but there's also a canonical way of overriding the default: you can use `.link` files to set up naming policies to suit your needs. Thus for instance if you have two PCs each of which has only a single wireless card, but one calls it `wlp0s1` and the other `wlp1s0`, you can arrange for them both to use the name `wifi0` to simplify sharing firewall configurations. For details see  [systemd.link\(5\)](#).

Here's a relatively futureproof "manual" version of the example [given above](#):

```
#/etc/systemd/network/10-persistent-net.link
[Match]
MACAddress=01:23:45:67:89:ab

[Link]
Name=lan0
```

Note: per  [systemd.link\(5\)](#), you shouldn't use a name that the kernel might use for another interface (for example "eth0").

It is also possible to reorganize the naming policy by overriding `/lib/systemd/network/99-default.link`, for instance to insist that all network interfaces are named purely by MAC address:

```
#/etc/systemd/network/99-default.link
[Match]
OriginalName=*

[Link]
NamePolicy=mac
MACAddressPolicy=persistent
```

EXTERNAL REFERENCES

 [Stretch](#) and  [buster](#) Release Notes

The nearest the upstream docs ever got to a canonical migration-HOWTO was <https://www.freedesktop.org/wiki/Software/systemd/PredictableNetworkInterfaceNames/>. The big problem with this was that it delegated all its technical details to a link pointing at the sourcecode: https://github.com/systemd/systemd/blob/master/src/udev/udev-builtin-net_id.c ...but most of the useful comments that used to be at the top of that file were then thrown out, so you need to find your way back through the `git` tree to a previous version such as https://github.com/systemd/systemd/blob/eefe36e64c1a583bb9470884ed92115e0ce4647e/src/udev/udev-builtin-net_id.c. Meanwhile, the page now claims to be obsolete, and points instead to <https://www.freedesktop.org/software/systemd/man/systemd.net-naming-scheme.html>, which is much less helpful. So the nearest thing left to an official HOWTO is probably `/usr/share/doc/udev/README.Debian.gz` (though it doesn't cover the "how to predict the names" part at all).

A guide that mentions `ID_NET_NAME_FROM_DATABASE`: <https://major.io/2015/08/21/understanding-systemds-predictable-network-device-names/>

General guides to overriding `systemd` configuration: <https://askubuntu.com/questions/659267/how-do-i-override-or-configure-systemd-services>, <https://wiki.archlinux.org/index.php/systemd>

[CategoryNetwork](#), [CategorySystemAdministration](#)

Keywords: persistent, predictable, NIC, wlan, eth, migrate

NetworkInterfaceNames (last modified 2020-06-05 12:22:26)

[Debian privacy policy](#).

[MoinMoin Powered](#) [Python Powered](#) Debian Wiki [team](#), [bugs](#) and [config](#) available. Hosting provided by [Metropolitan Area Network Darmstadt](#)